



| Nº     | NOME                              | E-mail                                  | Telefone       |
|--------|-----------------------------------|-----------------------------------------|----------------|
| 248037 | Bruno Bertolini Maluf             | brubemal@gmail.com                      | (15)99799-9909 |
| 235537 | Bruno Carvalho de Oliveira        | <u>brunocarvalho.ksc@gmail.com</u>      | (15)99192-4104 |
| 248245 | Guilherme Januário Serafim        | <u>guilhermeserafim.facul@gmail.com</u> | (15)99855-4581 |
| 248224 | Mateus Pasquotto Lulia Bellotto   | <u>Mateusplb27@gmail.com</u>            | (15)99820-8674 |
| 248312 | Rodrigo Lopes de Oliveira Cardoso | <u>lopesrodrigo942@gmail.com</u>        | (15)98807-4815 |

ATIVIDADE PRÁTICA:

(v1.0.0)

SOROCABA, SP

2026  
SUMÁRIO

|                                |   |
|--------------------------------|---|
| 1 CONCEITO DE VANILLA JS.....  | 2 |
| 2 VERSIONAMENTO E DEPLOY ..... | 5 |
| 3 GESTÃO DE PROJETO.....       | 7 |
| 4 PROJETO.....                 | 8 |
| REFERÊNCIAS.....               | 9 |

## 1 CRIAR UMA API

Antes de criar uma API, há de entender o que é uma API? **API** é a sigla para *Application Programming Interface* (Interface de Programação de Aplicações), definida como um **conjunto de regras, protocolos e definições** que permite a comunicação entre diferentes softwares ou sistemas. Ela atua como uma **ponte ou intermediário**, permitindo que aplicativos troquem dados e funcionalidades de forma padronizada e segura, sem que precisem conhecer os detalhes internos da implementação do outro.

O funcionamento de uma API baseia-se em uma interação simples de **solicitação e resposta** entre um **cliente** (quem pede a informação) e um **servidor** (quem a fornece). O processo segue estes passos:

- **Solicitação:** O aplicativo cliente envia uma requisição para a API, geralmente através de um endereço específico (endpoint) e usando métodos padrão como GET (obter dados) ou POST (enviar dados).
- **Processamento:** A API recebe a solicitação, verifica se está correta e se o cliente tem permissão de acesso, então encaminha o pedido ao sistema de destino.
- **Resposta:** O servidor processa a solicitação e envia uma resposta de volta para a API, que a entrega ao cliente, geralmente em formatos estruturados como **JSON** ou XML.

O **Back-end** é a parte do sistema que processa os dados e as regras de negócio "atrás das cortinas". Nesta tarefa, criaremos uma **API** (uma interface que permite que outros programas conversem entre si) utilizando **Node.js** e o framework **Express**.

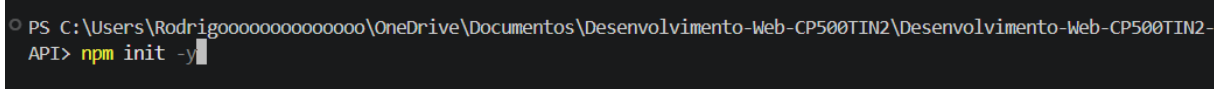
### 1. Preparação do Ambiente e Inicialização

Antes de começar, é necessário ter o **Node.js** instalado no computador. O Node permite que o JavaScript (linguagem comum nos navegadores) rode diretamente no sistema operacional.

- **Criação da Pasta:** Crie uma pasta exclusiva para o projeto (ex: projeto-api).

- **Inicialização do Projeto:** Abra o terminal dentro dessa pasta e digite: `npm init -y` como mostrado na imagem 1.

Imagem 1 - Inicialização do Projeto.



```
PS C:\Users\Rodrigoooooooooooo\OneDrive\Documentos\Desenvolvimento-Web-CP500TIN2\Desenvolvimento-Web-CP500TIN2-API> npm init -y
```

Fonte: Próprio Autor 2026.

Esse comando cria o arquivo `package.json`, que funciona como uma "certidão de nascimento" do projeto, listando suas dependências e configurações.

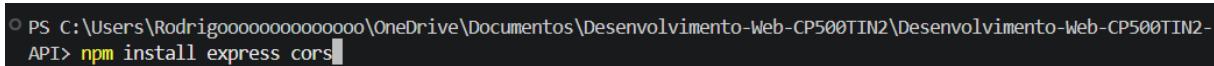
## 2. Instalação das Ferramentas (Dependências)

Para que a API funcione de forma profissional e acessível, precisamos instalar dois pacotes:

1. **Express:** O framework que facilita a criação de rotas (endereço da web).
2. **CORS:** Sigla para *Cross-Origin Resource Sharing*. Ele é essencial para dar "permissão" para que o seu Front-end (que estará em outro endereço) consiga consultar os dados da sua API sem ser bloqueado por segurança.

No terminal, execute: `npm install express cors`, como mostrado na imagem 2.

Imagem 2 - Instalação das ferramentas.



```
PS C:\Users\Rodrigoooooooooooo\OneDrive\Documentos\Desenvolvimento-Web-CP500TIN2\Desenvolvimento-Web-CP500TIN2-API> npm install express cors
```

Fonte: Próprio Autor 2026.

## 3. Construção do Servidor (`index.js`)

Crie um arquivo chamado `index.js`. Este será o coração da sua API. O código deve seguir esta estrutura explicada:

Imagem 3 - Código da API.

```
// 1. Importamos as ferramentas instaladas
const express = require('express');
const cors = require('cors');

// 2. Criamos a instância do servidor
const app = express();

// 3. Ativamos o CORS para permitir acessos externos
app.use(cors());

// 4. Definimos a porta onde o servidor vai rodar.
// No Render, ele usa uma variável de ambiente (process.env.PORT)
const PORT = process.env.PORT || 3000;

// 5. Criamos a "Rota": o endereço que será consultado
app.get('/data-hora', (req, res) => {
  const agora = new Date();

  // O comando res.json envia os dados formatados para o solicitante
  res.json({
    data: agora.toLocaleDateString('pt-BR'),
    hora: agora.toLocaleTimeString('pt-BR')
  });
});

// 6. Iniciamos o servidor para que ele fique "ouvindo" as chamadas
app.listen(PORT, () => {
  console.log(`Servidor ativo na porta ${PORT}`);
});
```

Fonte: Próprio Autor 2026.

#### 4. Versionamento com Git e GitHub

Para que o serviço de hospedagem (Render) consiga ler seu código, ele precisa estar em um repositório online.

1. Crie um repositório no **GitHub**.
2. Envie seus arquivos (exceto a pasta node\_modules, que é pesada e gerada automaticamente).
3. Certifique-se de que o arquivo package.json possui o campo "main": "index.js".

#### 5. Deploy no Render (Hospedagem)

O **Render** é o serviço que manterá seu código rodando na nuvem 24 horas por dia.

1. Faça login no [Render.com](https://render.com) e clique em **"New" > "Web Service"**.
2. Conecte sua conta do GitHub e selecione o repositório da API que você acabou de criar.
3. Nas configurações:

- **Runtime:** Node
  - **Build Command:** npm install (instala as dependências na nuvem).
  - **Start Command:** node index.js (executa seu servidor).
4. Clique em "**Deploy Web Service**". Após alguns minutos, o Render fornecerá um link (ex: <https://sua-api.onrender.com>).

## 6. Validação Final

Para testar se tudo está correto, pegue a URL gerada e adicione /data-hora ao final no seu navegador. Se aparecer um texto no formato JSON com a data e hora atual, sua API está pronta para ser consumida pelo Front-end.

## 2 RESULTADOS

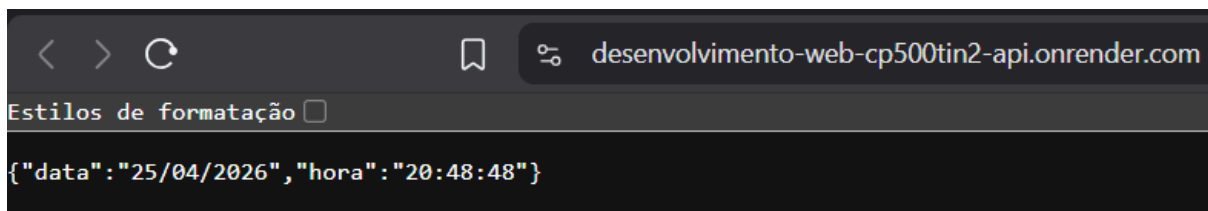
Após a conclusão das etapas de desenvolvimento e configuração, os seguintes resultados foram alcançados:

### 1. Disponibilidade da API (Back-end)

A API foi desenvolvida com sucesso utilizando **Node.js** e **Express**. Ela encontra-se hospedada na plataforma **Render**, o que garante que o serviço esteja disponível online de forma contínua.

- **Endpoint de consulta:** Ao acessar a rota /data-hora, o servidor processa a requisição e retorna um objeto em formato **JSON** como mostrado na imagem 4.

Imagem 4 – Endpoint.



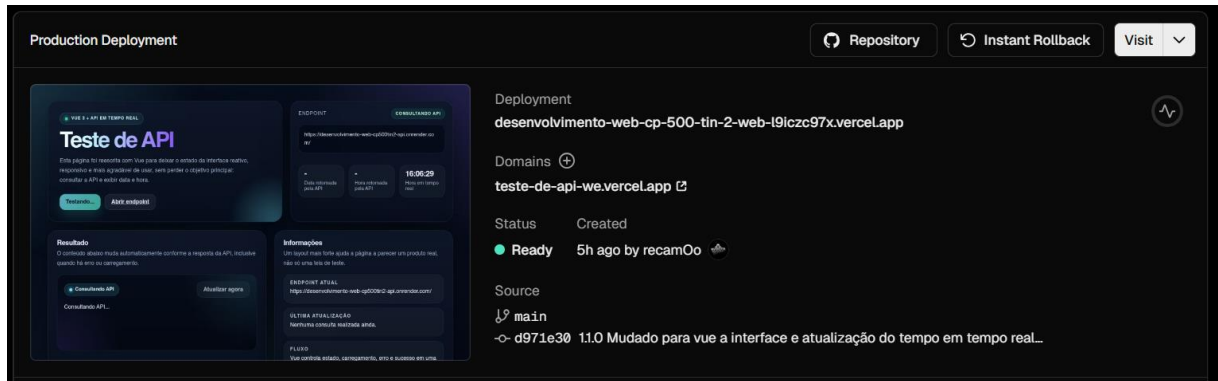
Fonte: Próprio Autor 2026.

Isso prova que a lógica de captura da data do sistema e a formatação para o padrão brasileiro (pt-BR) estão funcionando corretamente.

### 2. Interface do Usuário (Front-end)

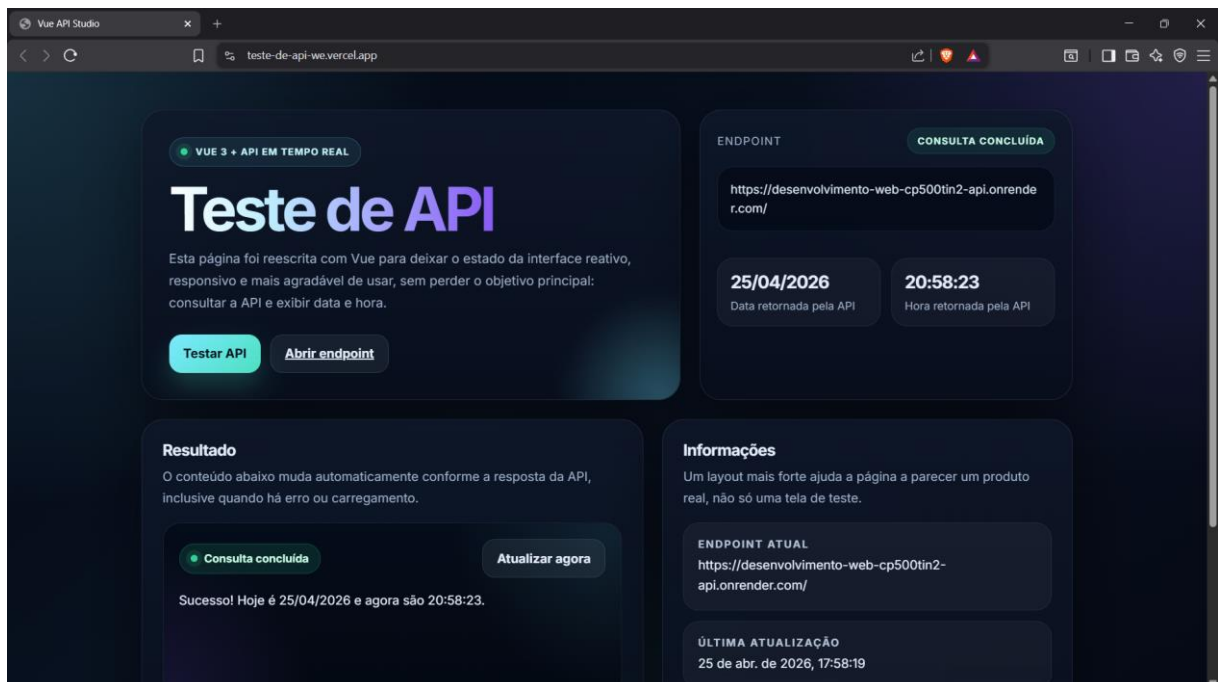
A aplicação Front-end foi criada de forma independente e hospedada na **Vercel** como mostrado na imagem 5 e 6. O resultado visual é uma página web simples e funcional que:

Imagem 5 – Aplicação na versel.



Fonte: Próprio Autor 2026.

Imagem 6 – Aplicação na Web.



Fonte: Próprio Autor 2026.

- Realiza uma chamada automática para a API assim que é carregada.
- Trata os dados recebidos e os apresenta de forma legível para o usuário final (ex: "Hoje é 25/04/2026 e agora são 14:30:05").

- Apresenta um design responsivo, podendo ser acessada tanto por computadores quanto por dispositivos móveis.

### 3. Integração e Comunicação entre Sistemas

Um dos resultados mais importantes foi a quebra da barreira de segurança **CORS** (Cross-Origin Resource Sharing). Através da configuração correta no Back-end, foi possível permitir que dois servidores diferentes (Render e Vercel) trocassem informações de maneira segura e eficiente.

### 4. Ciclo de Deploy Automatizado

A conexão dos repositórios do **GitHub** com as plataformas de hospedagem resultou em um fluxo de trabalho profissional. Qualquer alteração feita no código e enviada ao GitHub é automaticamente refletida no site ao vivo, demonstrando o uso de conceitos modernos de **CI/CD** (Integração Contínua e Entrega Contínua).

O projeto finalizou com dois sistemas distintos (um para dados e outro para visualização) trabalhando em harmonia. O objetivo de criar uma arquitetura **Client-Server** (Cliente-Servidor) funcional foi plenamente atingido, resultando em uma aplicação web completa e acessível através de uma URL pública.

## 3 PROJETO

- Projeto
  - GitHub: <https://github.com/recamOo/Desenvolvimento-Web-CP500TIN2-WEB-API>
  - GitHub: <https://github.com/recamOo/Desenvolvimento-Web-CP500TIN2-API>
  - Vercel: <https://teste-de-api-we.vercel.app/>
  - Render: <https://desenvolvimento-web-cp500tin2-api.onrender.com/>

## REFERÊNCIAS

- GIT. Documentação oficial. Disponível em: <https://git-scm.com/>. Acesso em: 2026.
- GITHUB. Plataforma de hospedagem de código. Disponível em: <https://github.com/>. Acesso em: 2026.
- VERCEL. Plataforma de deploy de aplicações. Disponível em: <https://vercel.com/>. Acesso em: 2026.